



Predict, Then Find

Predict the goal, run, find the bug

Name: _____

Date: _____

★ I can predict the right answer, run the code, and find the script that does not match.

Predict first, then check

Good debuggers know what the answer SHOULD be before they run. Bit (green) and Pixel (purple) both start on the green flag and should BOTH reach number 4. First PREDICT where each should land. Then run the code and find the one script that does NOT match your prediction.

The goal: both reach 4



Bit's script

🚩 when green flag clicked

forward 2

Pixel's script

🚩 when green flag clicked

forward 2

1 Predict the goal

The goal is for both to reach 4. Before running, what number SHOULD each sprite land on? Bit: _____

Pixel: _____

2 Run and find the off one

Now run both. Bit (home 2, forward 2): ____ Pixel (home 1, forward 2): _____. Which sprite did NOT match your prediction?

3 Find and fix

What is the bug in the off script, and what is the fix? Pixel starts at 1 and needs 4, so its move should be forward _____. After the fix, does it match the goal?

Big idea

When you predict the answer first, a bug is easy to spot: the script whose result does NOT match your prediction is the buggy one. Then you only have to find and fix that one. Here Pixel needed forward 3, not forward 2.



Predict, Then Find

Quick facilitation notes & answer key

What this worksheet teaches

Students combine predicting with debugging: predict what both SHOULD land on, run it, and use the mismatch to find the bug. The set whose run does not match the prediction is the buggy one. No coding experience needed.

Before you start (no computer needed)

No computer needed — paper and a pencil. You read the prompts aloud. Optional: two counters on a number line 0 to 5 (green Bit, purple Pixel) so children can trace both sets of moves and see which one misses the goal.

How to lead it (about 20 minutes)

1. Show them (you do). The goal is the yardstick — predict both reach 4, then run and compare.
2. Do one together. Exercise 1 — both should land on 4; Exercise 2 — Bit matches (4), Pixel does not (3).
3. Let them try. Students name the bug and the fix (Ex 3).

Answer key (these are the verified correct answers)

Exercise 1 — both should land on the goal 4.

Exercise 2 — Bit: 2, forward 2 → 4 (matches). Pixel: 1, forward 2 → 3 (does not match). Pixel is the off one.

Exercise 3 — Pixel starts at 1 and needs 4, so its move should be forward 3. After the fix, Pixel lands on 4 and matches the goal.

If students get stuck — and ways to adjust

Watch for: skipping the prediction. The prediction is what tells you which run is wrong.

Make it easier: write the predicted number above each character before running.

Make it harder: ask them to predict both AND name the likely bug before running.

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential and concurrent events.

In plain terms: students write and run instructions, including two sets that run at the same time.

C3.2 — read and alter existing code, including code that involves sequential and concurrent events, and describe how changes to the code affect the outcomes.

In plain terms: students read two sets of instructions, find and fix the wrong one, and explain the fix.

You'll know it worked when

The child predicts the goal, runs, finds the off set, and fixes it.